

Cours

Machine Learning for

Quantum Computing

Cours #4 – Logistic Regression

Professor
Abdelouahab Moussaoui

1 — Classification

Introduction

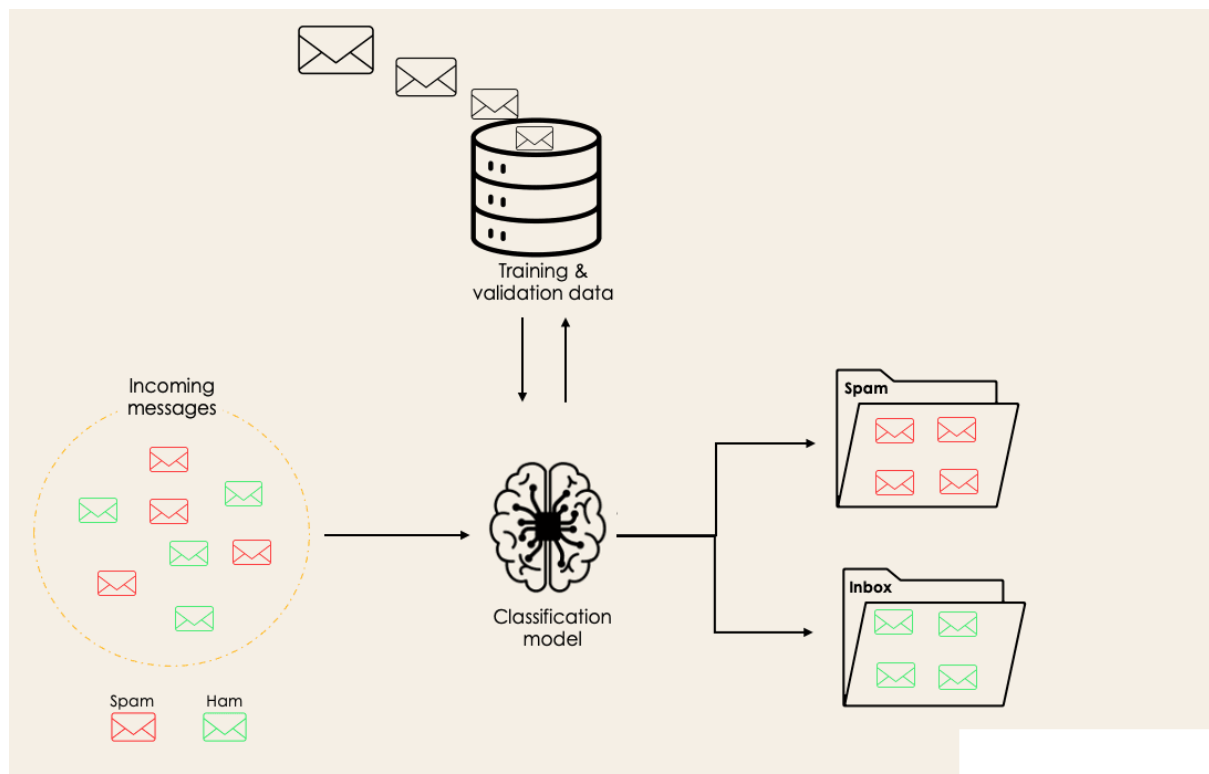
Nowadays, many industries have been dealing with very large data sets of different types. Manually processing all that information can be time-consuming and might not even add value in the long term. Many strategies, from simple automation to machine learning techniques, are being applied for a better return on investment. This conceptual blog will cover one of the most important concepts; classification in machine learning.

We will start by defining what classification is in Machine Learning before clarifying the two types of learners in machine learning and the difference between classification and regression. Then, we will cover some real-world scenarios where classification can be used. After that, we will introduce all the different types of classification and deep dive into some examples of classification algorithms. Finally, we will provide hands-on practice on the implementation of a few algorithms.

What is Classification in Machine Learning?

Classification is a supervised machine learning method where the model tries to predict the correct label of a given input data. In classification, the model is fully trained using the training data, and then it is evaluated on test data before being used to perform prediction on new unseen data.

For instance, an algorithm can learn to predict whether a given email is spam or ham (no spam), as illustrated below.



Before diving into the classification concept, we will first understand the difference between the two types of learners in classification: lazy and eager learners. Then we will clarify the misconception between classification and regression.

Lazy Learners Vs. Eager Learners

There are two types of learners in machine learning classification: lazy and eager learners.

Eager learners are machine learning algorithms that first build a model from the training dataset before making any prediction on future datasets. They spend more time during the training process because of their eagerness to have a better generalization during the training from learning the weights, but they require less time to make predictions.

Most machine learning algorithms are eager learners, and below are some examples:

- *Logistic Regression.*
- *Support Vector Machine.*
- *Decision Trees.*
- *Artificial Neural Networks.*

Lazy learners or instance-based learners, on the other hand, do not create any model immediately from the training data, and this is where the lazy aspect comes from. They just memorize the training data, and each time there is a need to make a prediction, they search for the nearest neighbor from the whole training data, which makes them very slow during prediction. Some examples of this kind are:

- *K-Nearest Neighbor.*
- *Case-based reasoning.*

However, some algorithms, such as [BallTrees](#) and [KDTrees](#), can be used to improve the prediction latency.

Machine Learning Classification Vs. Regression

There are four main categories of Machine Learning algorithms: supervised, unsupervised, semi-supervised, and reinforcement learning.

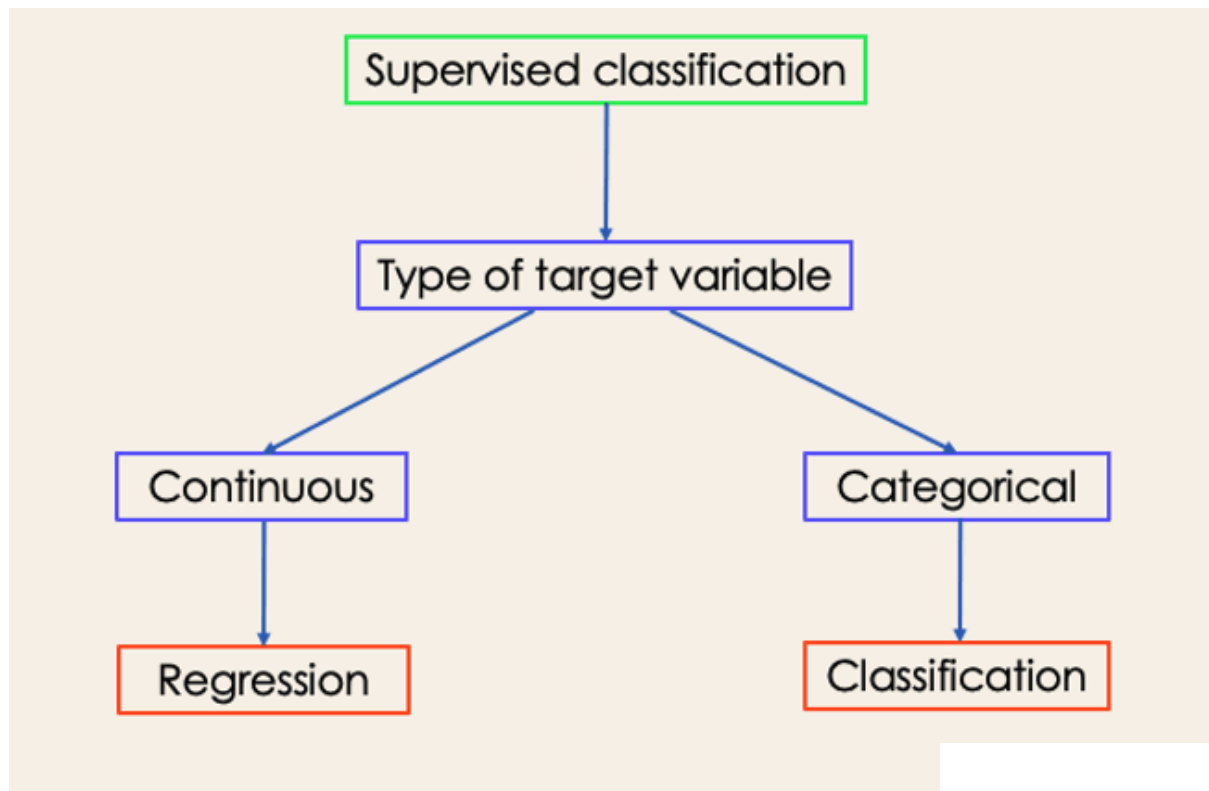
Even though classification and regression are both from the category of supervised learning, they are not the same.

- The prediction task is a **classification** when the target variable is discrete. An application is the identification of the underlying sentiment of a piece of text.
- The prediction task is a **regression** when the target variable is continuous. An example can be the prediction of the salary of a person given their education degree, previous work experience, geographical location, and level of seniority.

If you are interested in knowing more about classification, courses on [Supervised Learning with scikit-learn](#) and [Supervised Learning in R might be helpful](#). They provide you with a better understanding of how each algorithm approaches tasks and the Python and R functions required to implement them.

Regarding regression, [Introduction to Regression in R](#) and [Introduction to Regression with statsmodels in Python](#) will help you explore different types of regression models as well as their implementation in R and Python.

Examples of Machine Learning Classification in Real Life



Supervised Machine Learning Classification has different applications in multiple domains of our day-to-day life. Below are some examples.

Healthcare

Training a machine learning model on historical patient data can help healthcare specialists accurately analyze their diagnoses:

- During the COVID-19 pandemic, machine learning models were implemented to efficiently predict whether a person had COVID-19 or not.
- Researchers can use machine learning models to predict new diseases that are more likely to emerge in the future.

Education

Education is one of the domains dealing with the most textual, video, and audio data. This unstructured information can be analyzed with the help of Natural Language technologies to perform different tasks such as:

- The classification of documents per category.
- Automatic identification of the underlying language of students' documents during their application.
- Analysis of students' feedback sentiments about a Professor.

Transportation

Transportation is the key component of many countries' economic development. As a result, industries are using machine and deep learning models:

- To predict which geographical location will have a rise in traffic volume.
- Predict potential issues that may occur in specific locations due to weather conditions.

Sustainable agriculture

Agriculture is one of the most valuable pillars of human survival. Introducing sustainability can help improve farmers' productivity at a different level without damaging the environment:

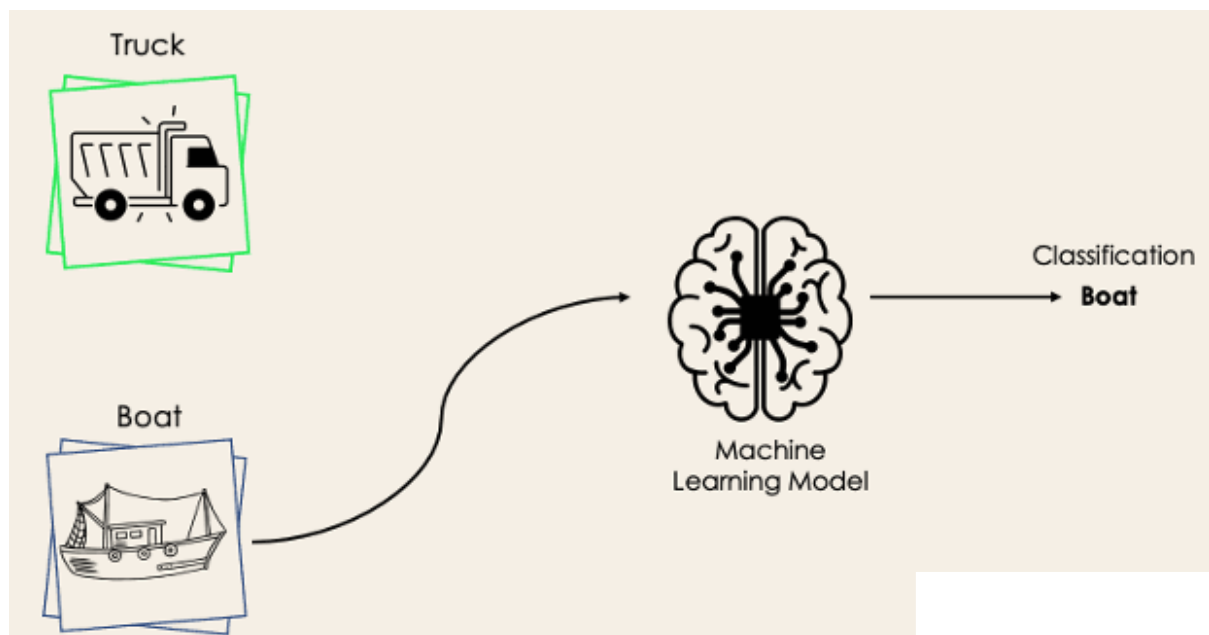
- By using classification models to predict which type of land is suitable for a given type of seed.
- Predict the weather to help them take proper preventive measures.

Different Types of Classification Tasks in Machine Learning

There are four main classification tasks in Machine learning: binary, multi-class, multi-label, and imbalanced classifications.

Binary Classification

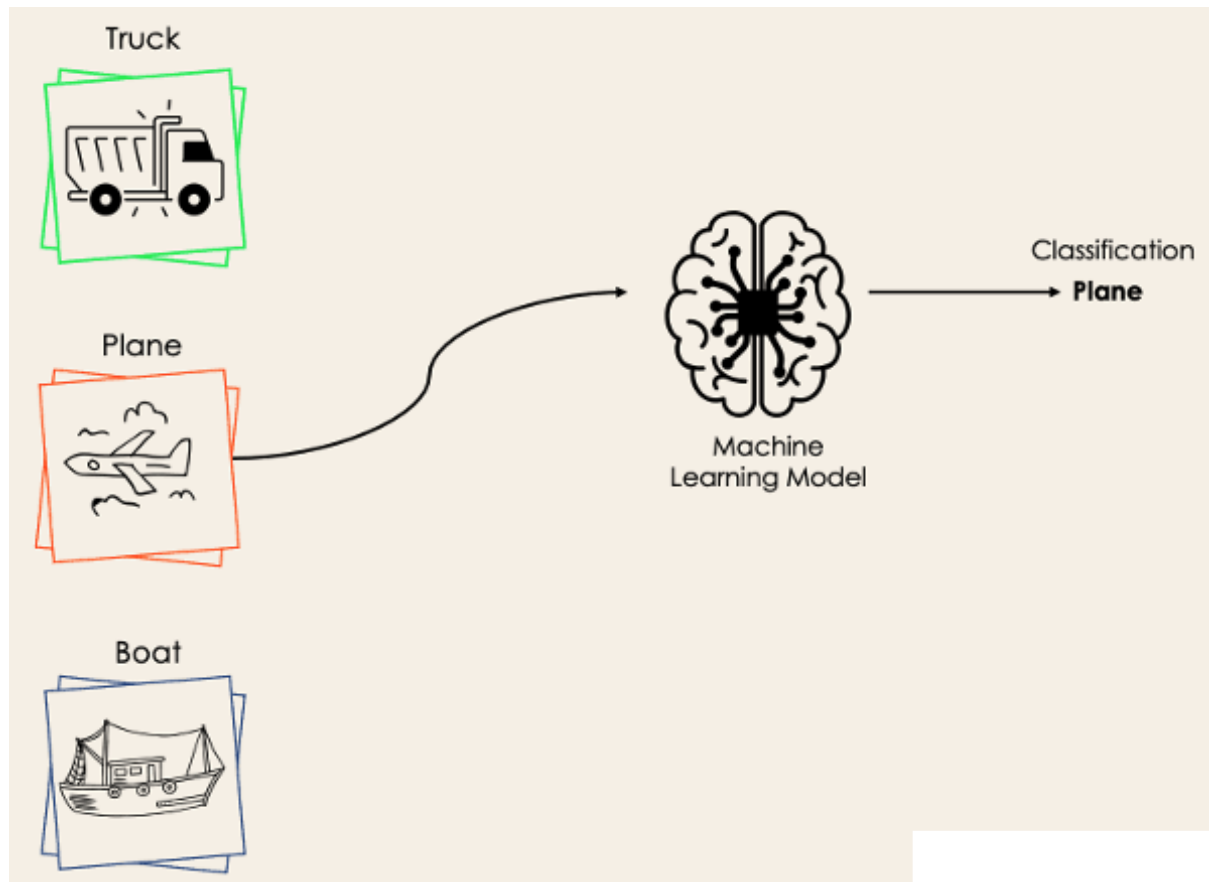
In a binary classification task, the goal is to classify the input data into two mutually exclusive categories. The training data in such a situation is labeled in a binary format: true and false; positive and negative; 0 and 1; spam and not spam, etc. depending on the problem being tackled. For instance, we might want to detect whether a given image is a truck or a boat.



Logistic Regression and Support Vector Machines algorithms are natively designed for binary classifications. However, other algorithms such as K-Nearest Neighbors and Decision Trees can also be used for binary classification.

Multi-Class Classification

The multi-class classification, on the other hand, has at least two mutually exclusive class labels, where the goal is to predict to which class a given input example belongs to. In the following case, the model correctly classified the image to be a plane.



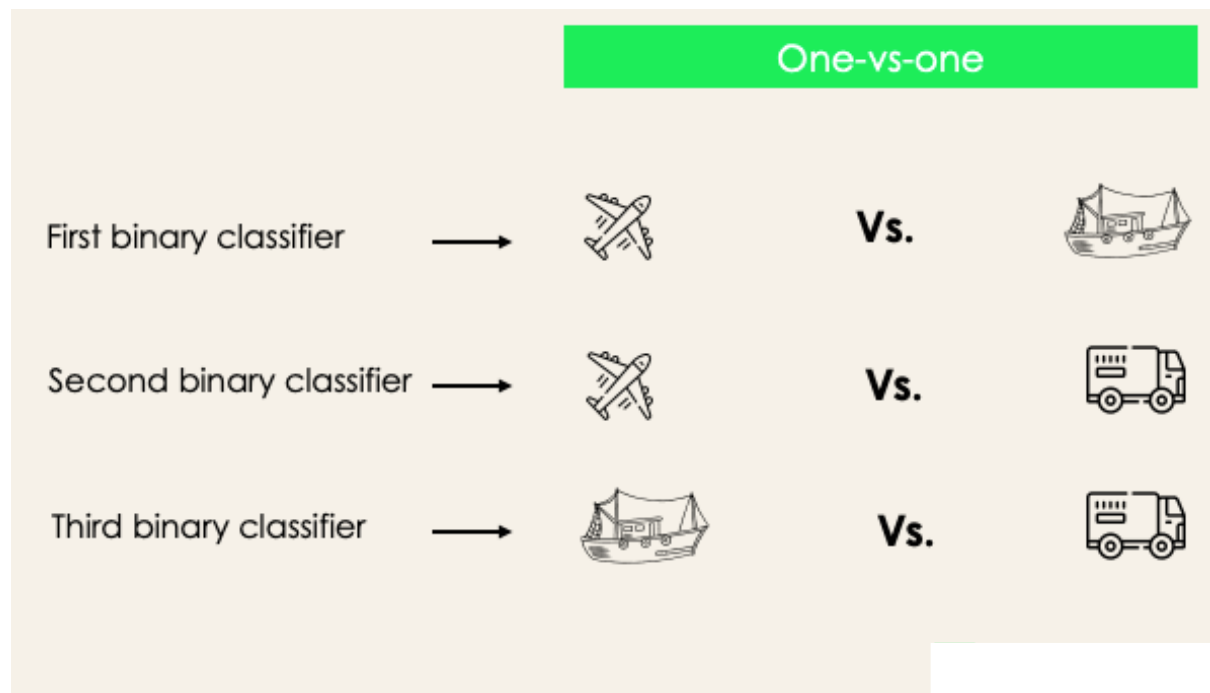
Most of the binary classification algorithms can be also used for multi-class classification. These algorithms include but are not limited to:

- *Random Forest*
- *Naive Bayes*
- *K-Nearest Neighbors*
- *Gradient Boosting*
- *SVM*
- *Logistic Regression.*

But wait! Didn't you say that SVM and Logistic Regression do not support multi-class classification by default?

→ That's correct. However, we can apply binary transformation approaches such as one-versus-one and one-versus-all to adapt native binary classification algorithms for multi-class classification tasks.

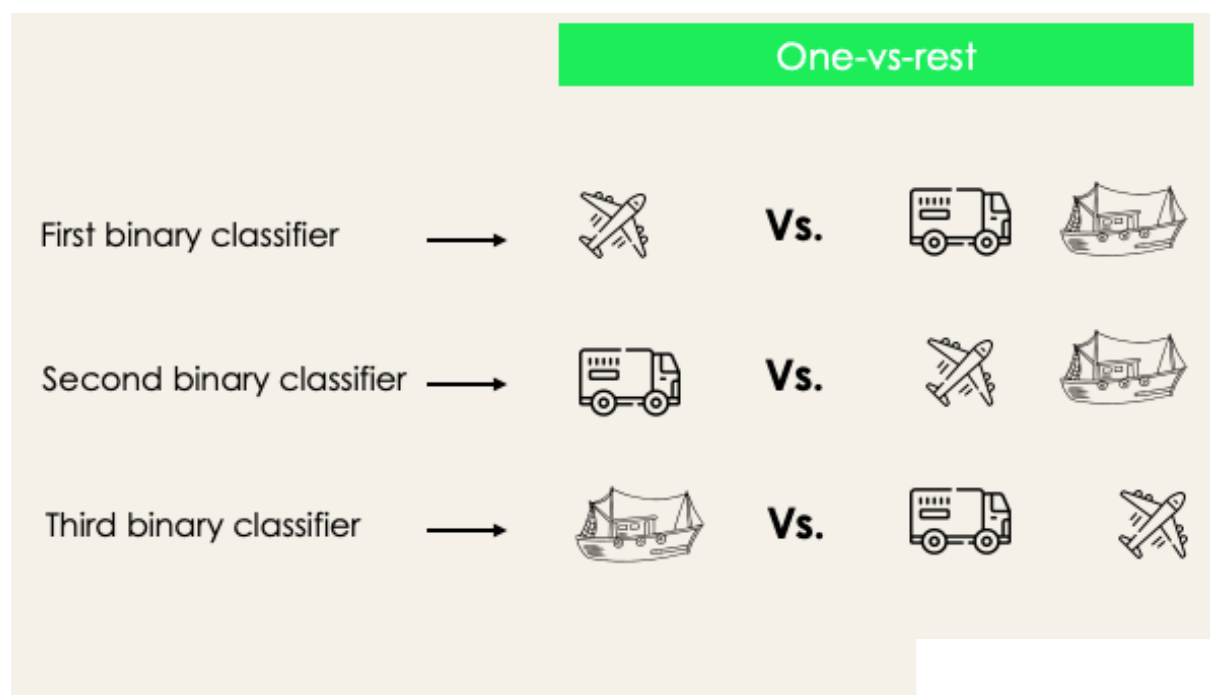
One-versus-one: this strategy trains as many classifiers as there are pairs of labels. If we have a 3-class classification, we will have three pairs of labels, thus three classifiers, as shown below.



In general, for N labels, we will have $N \times (N-1)/2$ classifiers. Each classifier is trained on a single binary dataset, and the final class is predicted by a majority vote between all the classifiers. One-vs-one approach works best for SVM and other kernel-based algorithms.

One-versus-rest: at this stage, we start by considering each label as an independent label and consider the rest combined as only one label. With 3-classes, we will have three classifiers.

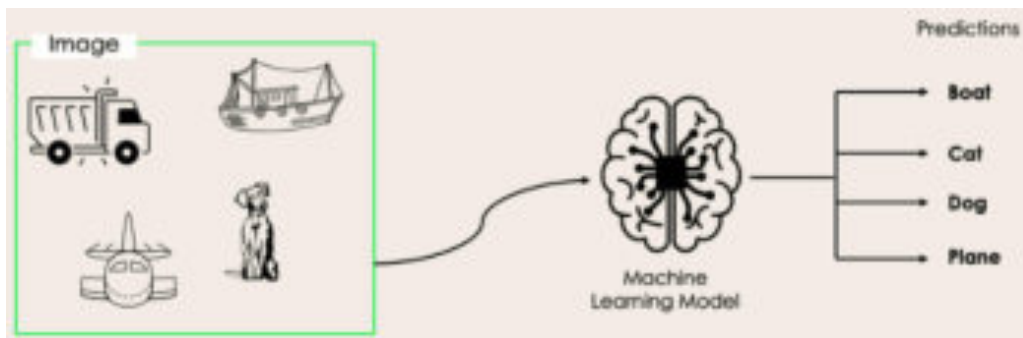
In general, for N labels, we will have N binary classifiers.



Multi-Label Classification

In multi-label classification tasks, we try to predict 0 or more classes for each input example. In this case, there is no mutual exclusion because the input example can have more than one label.

Such a scenario can be observed in different domains, such as auto-tagging in Natural Language Processing, where a given text can contain multiple topics. Similarly to computer vision, an image can contain multiple objects, as illustrated below: the model predicted that the image contains: a plane, a boat, a truck, and a dog.

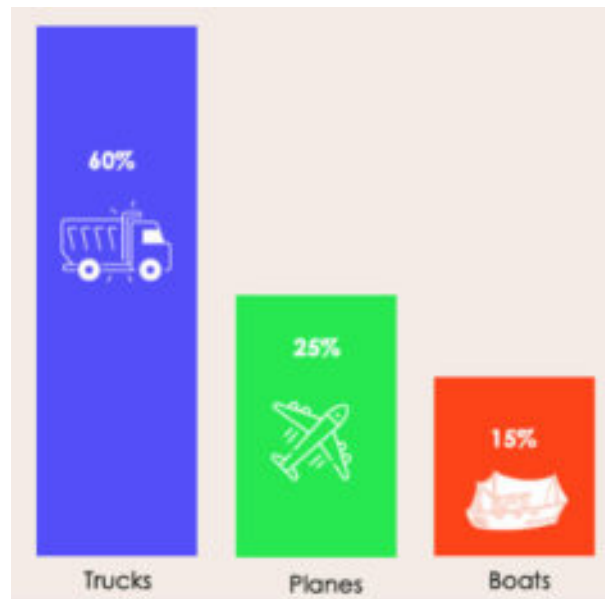


It is not possible to use multi-class or binary classification models to perform multi-label classification. However, most algorithms used for those standard classification tasks have their specialized versions for multi-label classification. We can cite:

- *Multi-label Decision Trees*
- *Multi-label Gradient Boosting*
- *Multi-label Random Forests*

Imbalanced Classification

For the imbalanced classification, the number of examples is unevenly distributed in each class, meaning that we can have more of one class than the others in the training data. Let's consider the following 3-class classification scenario where the training data contains: 60% of trucks, 25% of planes, and 15% of boats.



The imbalanced classification problem could occur in the following scenario:

- *Fraudulent transaction detections in financial industries*
- *Rare disease diagnosis*
- *Customer churn analysis*

Using conventional predictive models such as Decision Trees, Logistic Regression, etc. could not be effective when dealing with an imbalanced dataset, because they might be biased toward predicting the class with the highest number of observations, and considering those with fewer numbers as noise.

So, does that mean that such problems are left behind?

Of course not! We can use multiple approaches to tackle the imbalance problem in a dataset. The most commonly used approaches include sampling techniques or harnessing the power of cost-sensitive algorithms.

Sampling Techniques

These techniques aim to balance the distribution of the original by:

- *Cluster-based Oversampling:*
- *Random undersampling: random elimination of examples from the majority class.*
- *SMOTE Oversampling: random replication of examples from the minority class.*

Cost-Sensitive Algorithms

These algorithms take into consideration the cost of misclassification. They aim to minimize the total cost generated by the models.

- *Cost-sensitive Decision Trees.*
- *Cost-sensitive Logistic Regression.*
- *Cost-sensitive Support Vector Machines.*

Metrics to Evaluate Machine Learning Classification Algorithms

Now that we have an idea of the different types of classification models, it is crucial to choose the right evaluation metrics for those models. In this section, we will cover the most commonly used metrics: accuracy, precision, recall, F1 score, and area under the ROC (Receiver Operating Characteristic) curve and AUC (Area Under the Curve).

A bit of context

Imagine you are a healthcare startup, and want an AI assistant able to predict whether a given patient has a heart disease or not based on its health record. This is a binary classification problem where the model will predict

- 1, True or Yes if the patient has heart disease
- 0, False or No otherwise

1 Confusion matrix

A 2X2 matrix that nicely summarizes the number of correct predictions of the model. It also helps in computing different other performance metrics.

Predicti Reality	Yes	No
Yes	True Positives (TP)	False Negatives (FN)
No	False Positives (FP)	True Negatives (TN)

Type I Error
Type II Error

Type I & II Errors can be used interchangeably when referring to False Positives and False negatives respectively

2 Accuracy

We get accuracy by answering this question: "out of the predictions made by the model, what percentage is correct?"

$$\text{Accuracy} = \frac{TP + TN}{\text{Total number observation}}$$

3 Precision

We get precision by answering this question: "out of all the YES predictions, how many of them were correct?"

$$\text{Precision} = \frac{TP}{TP + FP}$$

4 Recall / Sensitivity

It aims to answer this question: "how good was the model at predicting real Yes events", which can be considered as the flip of the precision.

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

5 Recall / Specificity

It aims to answer this question: "how good was the model at predicting real No events".

$$\text{Specificity} = \frac{TN}{TN + FP}$$

6 F1 Score

Sometimes used when dealing with imbalanced data set, meaning that there are more of one class/label than there are of the other. It corresponds to the harmonic mean of the precision and recall.

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

7 AUC – ROC Curve

AUC– ROC generates probability values instead of binary 0/1 values. It should be used when your data set is roughly balanced.

Using ROC for imbalanced data sets lead to incorrect interpretation.

ROC curves provide good overview of trade-off between the TP rate and FP rate for binary classifier using different probability thresholds.

- A value below 0.5 indicates a poor classifier
- A value of 0.5 means random classifier
- Value over 0.7 corresponds to a good classifier
- 0.8 indicates a strong classifier
- We have 1 when the classifier perfectly predicts everything.

Strategies to choose the right metric**Choose accuracy**

- The cost of FP and FN are roughly equal.
- The benefit of TP and TN are roughly equal.

Choose Precision

- The cost of FP is much higher than a FN.
- The benefit of a TP is much higher than a TN.

Choose recall

- The cost of FN is much higher than a FP.
- The cost of a TN is much higher than a TP.

ROC AUC & Precision – Recall curves

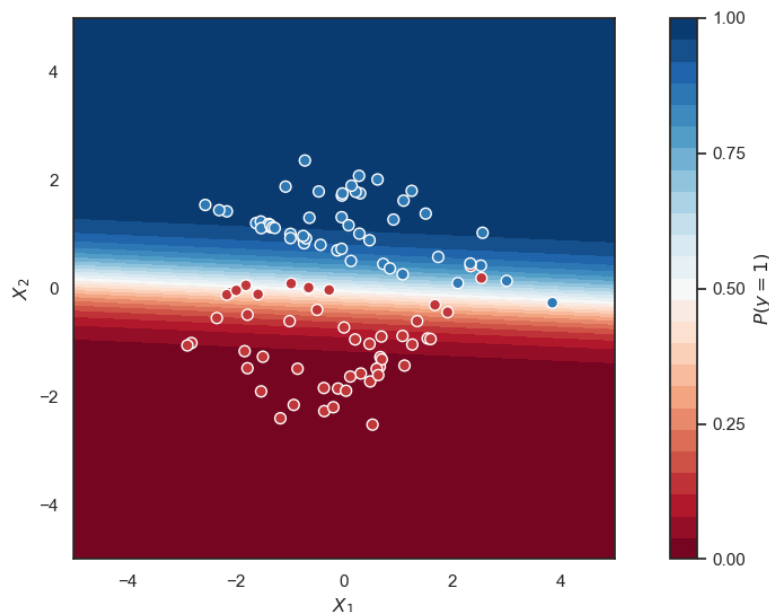
- Use ROC when the dealing with balanced data sets.
- Use precision-recall for imbalanced data sets.

Deep Dive into Classification Algorithms

We now have all the tools in hand to proceed with the implementation of some algorithms. This section will cover four algorithms and their implementation on the [loans dataset](#) to illustrate some of the previously covered concepts, especially for the imbalanced datasets using a binary classification task. We will focus on only four algorithms for simplicity's sake.

The goal is not to have the best possible model but to illustrate how to train each of the following algorithms. The source code is available on the [DataCamp Workspace](#), where you can execute everything with one click.

2 — Logistic Regression



Logistic Regression is a classical statistical model, which has been widely used in academia and industry to solve binary classification problems.

Difference between Regression and Classification

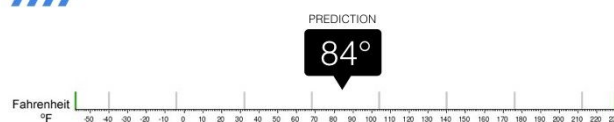
Supervised Machine Learning can be split into two subcategories – Regression and Classification.

The difference between the two is that in Regression we are predicting a continuous number like the price of a house or the temperature for the next day whilst in Classification, we are predicting discrete values like if a patient has or doesn't have a heart disease.



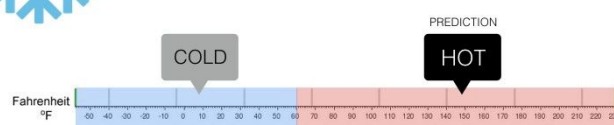
Regression

What is the temperature going to be tomorrow?



Classification

Will it be Cold or Hot tomorrow?



Logistic Regression Theory

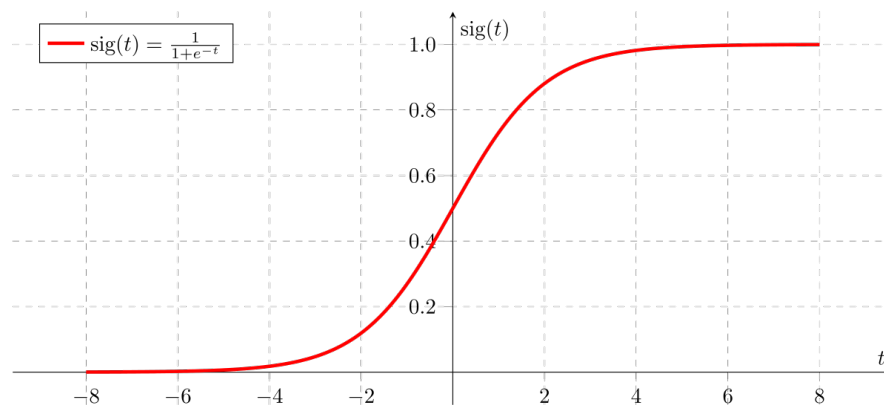
Logistic Regression is a statistical method that was designed to solve binary classification problems. It achieves this by passing the input through a linear function and then transforming the output to a probability value with the help of a [sigmoid function](#).

Mathematically this looks like:

$$h_{\theta}(x) = P(Y = 1|x; \theta) = \text{sigmoid}(Z)$$

$$Z = \theta^T x$$

Sigmoid Function



Without the Sigmoid function, Logistic Regression would just be Linear Regression. That means that the output of the model could range from $-\infty$ to ∞ .

That's fine when working on a regression task but for binary classification, the output needs to be a probability value. This is where the sigmoid function comes in. It squeezes the output of the linear function Z between 0 and 1. All input values greater than 0 produce an output greater than 0.5. All inputs less than 0 produce an output between less than 0.5.

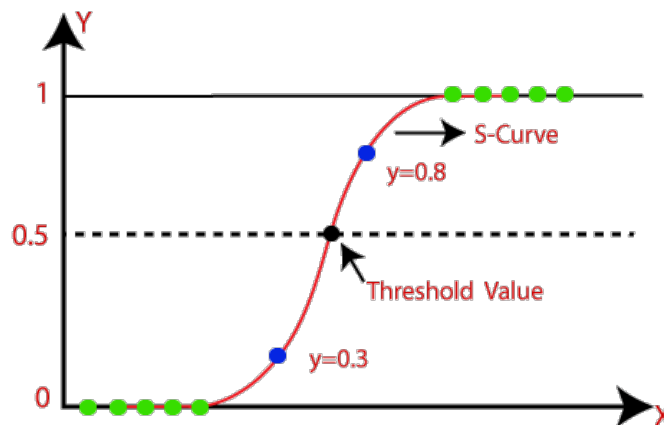
Mathematically the sigmoid function looks like:

$$\text{sigmoid}(Z) = \frac{1}{1+e^{-Z}}$$

Characteristics

- Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables.
- Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, **it gives the probabilistic values which lie between 0 and 1.**
- Logistic Regression is much similar to the Linear Regression except that how they are used. Linear Regression is used for solving Regression problems, whereas **Logistic regression is used for solving the classification problems.**

- In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).
- The curve from the logistic function indicates the likelihood of something such as whether the cells are cancerous or not, a mouse is obese or not based on its weight, etc.
- Logistic Regression is a significant machine learning algorithm because it has the ability to provide probabilities and classify new data using continuous and discrete datasets.
- Logistic Regression can be used to classify the observations using different types of data and can easily determine the most effective variables used for the classification. The below image is showing the logistic function:



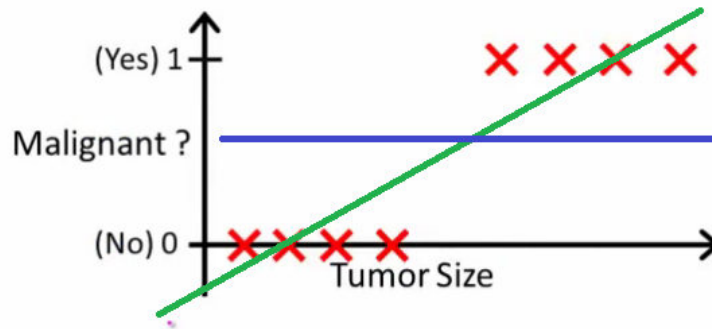
Note: Logistic regression uses the concept of predictive modeling as regression; therefore, it is called logistic regression, but is used to classify samples; Therefore, it falls under the classification algorithm.

Why do we use Logistic Regression rather than Linear Regression?

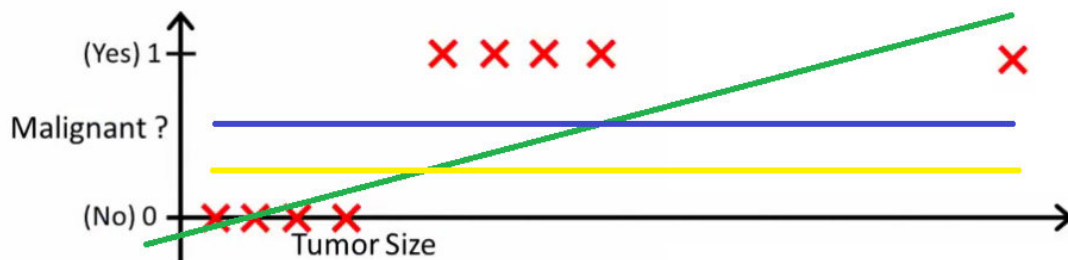
If you have this doubt, then you're in the right place, my friend. After reading the definition of logistic regression we now know that it is only used when our dependent variable is binary and in linear regression this dependent variable is continuous.

The second problem is that if we add an outlier in our dataset, the best fit line in linear regression shifts to fit that point.

Now, if we use linear regression to find the best fit line which aims at minimizing the distance between the predicted value and actual value, the line will be like this:



Here the threshold value is 0.5, which means if the value of $h(x)$ is greater than 0.5 then we predict malignant tumor (1) and if it is less than 0.5 then we predict benign tumor (0). Everything seems okay here but now let's change it a bit, we add some outliers in our dataset, now this best fit line will shift to that point. Hence the line will be somewhat like this:

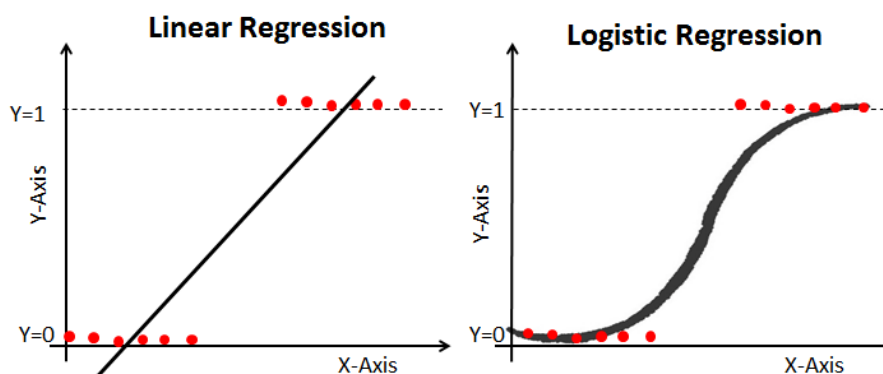


Do you see any problem here? The blue line represents the old threshold and the yellow line represents the new threshold which is maybe 0.2 here. To keep our predictions right we had to lower our threshold value. Hence we can say that linear regression is prone to outliers. Now here if $h(x)$ is greater than 0.2 then only this regression will give correct outputs.

Another problem with linear regression is that the predicted values may be out of range. We know that probability can be between 0 and 1, but if we use linear regression this probability may exceed 1 or go below 0.

To overcome these problems we use Logistic Regression, which converts this straight best fit line in linear regression to an S-curve using the sigmoid function, which will always give values between 0 and 1. How does this work and what's the math behind this will be covered in a later section?

Now we have our logistic function, also called a sigmoid function. The graph of a sigmoid function is as shown below. It squeezes a straight line into an S-curve.



Decision Boundary

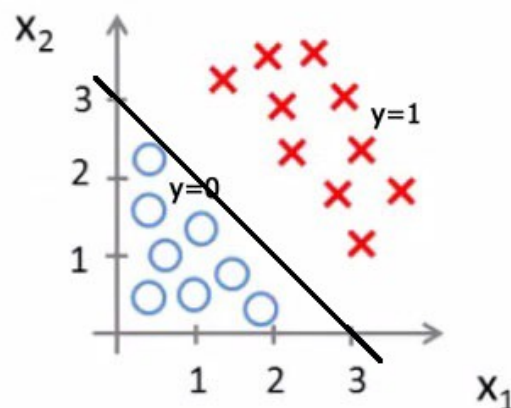
To get a discrete class value (either 0 or 1) a decision boundary must be chosen. The decision boundary specifies how high the probability must be so that we have an output of 1.

Generally, the decision boundary is 0.5 so that if the output is ≥ 0.5 we get class 1, else class 0.

$$h_{\theta}(x) \geq 0.5 \rightarrow y = 1$$

$$h_{\theta}(x) < 0.5 \rightarrow y = 0$$

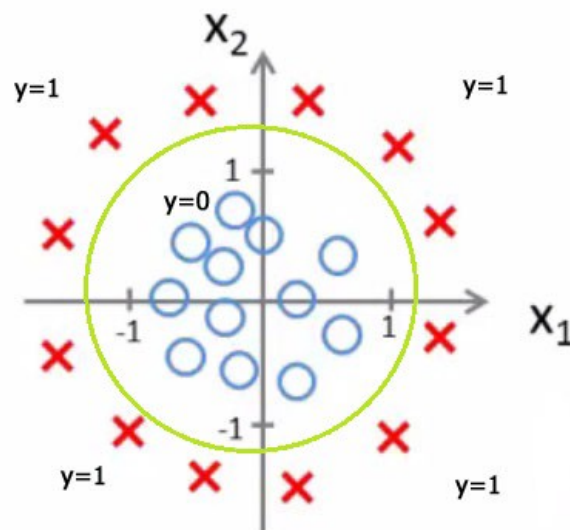
- Linear Decision Boundary



For $y = 1$, Equation of line would be $x_1 + x_2 \geq 3$

For $y = 0$, Equation of line would be $x_1 + x_2 < 3$

- Non-Linear Decision Boundary



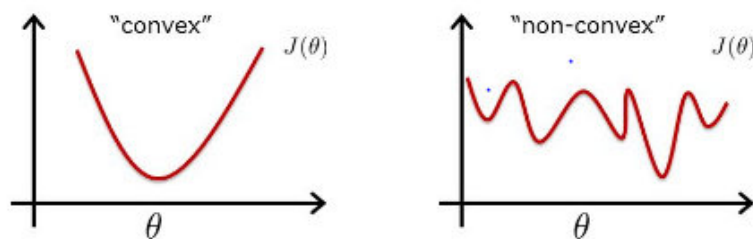
For $y=1$, equation would be $x_1^2 + x_2^2 \geq 1$

For $y=0$, equation would be $x_1^2 + x_2^2 < 1$

Loss Function

For Logistic Regression we can't use the same loss function as for Linear Regression because the Logistic Function (Sigmoid Function) will cause the output to be non-convex, which will cause many local optima.

Convex Vs Non-Convex

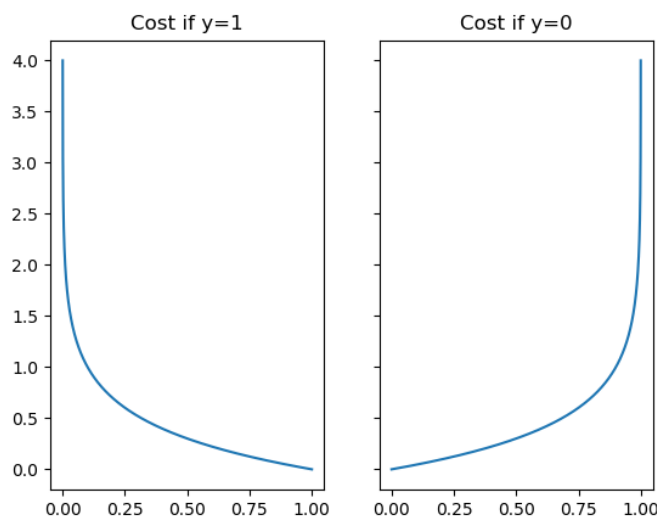


Instead, we will use the following loss function for logistic regression:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)})$$

$$\begin{aligned} \text{Cost}(h_{\theta}(x), y) &= -\log(h_{\theta}(x)) && \text{if } y = 1 \\ \text{Cost}(h_{\theta}(x), y) &= -\log(1 - h_{\theta}(x)) && \text{if } y = 0 \end{aligned}$$

At first glance, the functions look complex but when visualized they are quite easy to grasp.



The above graph shows that the further away the prediction is from the actual y value the bigger the loss gets.

That means that if the correct answer is 0, then the cost function will be 0 if the prediction is also 0. If the prediction approaches 1, then the cost function will approach infinity.

If the correct answer is 1, then the cost function will be 0 if the prediction is 1. If the prediction approaches 0, then the cost function will approach infinity.

Simplifying the Loss Function

To make it easier to work with the loss function we can compress the two conditional cases into one equation:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

Notice that when y is equal to 1 the second term will be zero and therefore will not affect the loss.

On the other hand if y is equal to 0 the first term will be zero and therefore will not affect the loss.

Vectorized this looks like:

$$J(\theta) = \frac{1}{m} \cdot (-y^T \log(h) - (1 - y)^T \log(1 - h))$$

Gradient Descent

To find the coefficients (weights) that minimize the loss function we will use Gradient Descent. There are more sophisticated optimization algorithms out there such as [Adam](#) but we won't worry about those in this article.

Remember the general form of gradient descent looks like:

$$\begin{aligned} &\text{Repeat } \{ \\ &\quad \theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \\ &\} \end{aligned}$$

We can get the gradient descent formula for Logistic Regression by taking the derivative of the loss function. This is quite involved therefore I will show you the result first and you can skip the process of getting to the result if you like.

Result:

$$\begin{aligned} &\text{Repeat } \{ \\ &\quad \theta_j := \theta_j - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \\ &\} \end{aligned}$$

Notice that the result is identical to the one of Linear Regression.

Deriving the Gradient Descent formula for Logistic Regression (Optional)

First we need to calculate the derivative of the sigmoid function. The derivative of the sigmoid function is quite easy to calculate using [the quotient rule](#).

$$\begin{aligned} \sigma(x)' &= \left(\frac{1}{1 + e^{-x}} \right)' = \frac{-(1 + e^{-x})'}{(1 + e^{-x})^2} = \frac{-1' - (e^{-x})'}{(1 + e^{-x})^2} = \frac{0 - (-x)'(e^{-x})}{(1 + e^{-x})^2} = \frac{-(-1)(e^{-x})}{(1 + e^{-x})^2} = \frac{e^{-x}}{(1 + e^{-x})^2} \\ &= \left(\frac{1}{1 + e^{-x}} \right) \left(\frac{e^{-x}}{1 + e^{-x}} \right) = \sigma(x) \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) = \sigma(x) \left(\frac{1 + e^{-x}}{1 + e^{-x}} - \frac{1}{1 + e^{-x}} \right) = \sigma(x)(1 - \sigma(x)) \end{aligned}$$

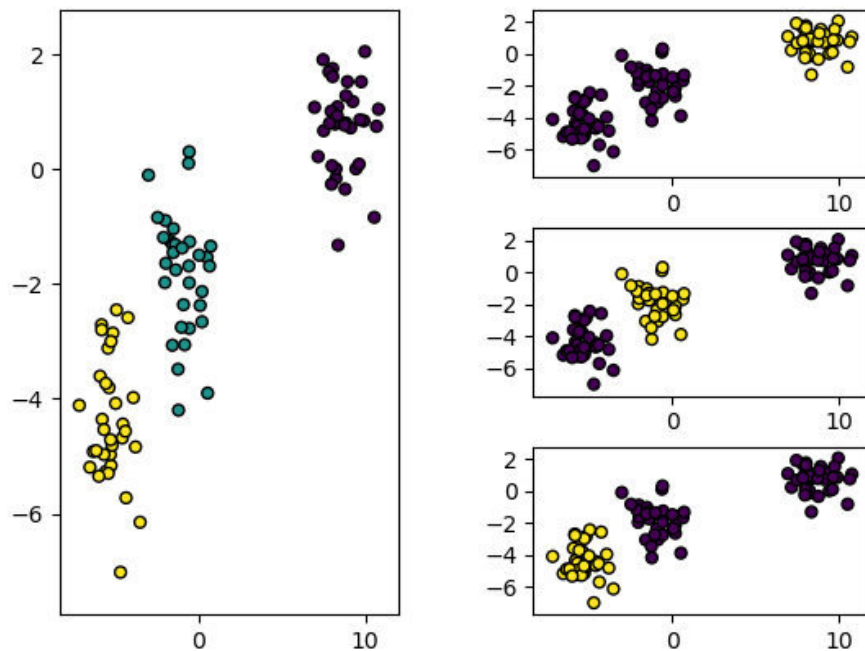
Now we are ready to find out the partial derivative:

$$\begin{aligned}
 \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{-1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right] \\
 &= -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \frac{\partial}{\partial \theta_j} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \frac{\partial}{\partial \theta_j} \log(1 - h_\theta(x^{(i)})) \right] \\
 &= -\frac{1}{m} \sum_{i=1}^m \left[\frac{y^{(i)} \frac{\partial}{\partial \theta_j} h_\theta(x^{(i)})}{h_\theta(x^{(i)})} + \frac{(1 - y^{(i)}) \frac{\partial}{\partial \theta_j} (1 - h_\theta(x^{(i)}))}{1 - h_\theta(x^{(i)})} \right] \\
 &= -\frac{1}{m} \sum_{i=1}^m \left[\frac{y^{(i)} \frac{\partial}{\partial \theta_j} \sigma(\theta^T x^{(i)})}{h_\theta(x^{(i)})} + \frac{(1 - y^{(i)}) \frac{\partial}{\partial \theta_j} (1 - \sigma(\theta^T x^{(i)}))}{1 - h_\theta(x^{(i)})} \right] \\
 &= -\frac{1}{m} \sum_{i=1}^m \left[\frac{y^{(i)} \sigma(\theta^T x^{(i)}) (1 - \sigma(\theta^T x^{(i)})) \frac{\partial}{\partial \theta_j} \theta^T x^{(i)}}{h_\theta(x^{(i)})} + \frac{-(1 - y^{(i)}) \sigma(\theta^T x^{(i)}) (1 - \sigma(\theta^T x^{(i)})) \frac{\partial}{\partial \theta_j} \theta^T x^{(i)}}{1 - h_\theta(x^{(i)})} \right] \\
 &= -\frac{1}{m} \sum_{i=1}^m \left[\frac{y^{(i)} h_\theta(x^{(i)}) (1 - h_\theta(x^{(i)})) \frac{\partial}{\partial \theta_j} \theta^T x^{(i)}}{h_\theta(x^{(i)})} - \frac{(1 - y^{(i)}) h_\theta(x^{(i)}) (1 - h_\theta(x^{(i)})) \frac{\partial}{\partial \theta_j} \theta^T x^{(i)}}{1 - h_\theta(x^{(i)})} \right] \\
 &= -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} (1 - h_\theta(x^{(i)})) x_j^{(i)} - (1 - y^{(i)}) h_\theta(x^{(i)}) x_j^{(i)} \right] \\
 &= -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} (1 - h_\theta(x^{(i)})) - (1 - y^{(i)}) h_\theta(x^{(i)}) \right] x_j^{(i)} \\
 &= -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} - y^{(i)} h_\theta(x^{(i)}) - h_\theta(x^{(i)}) + y^{(i)} h_\theta(x^{(i)}) \right] x_j^{(i)} \\
 &= -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} - h_\theta(x^{(i)}) \right] x_j^{(i)} \\
 &= \frac{1}{m} \sum_{i=1}^m \left[h_\theta(x^{(i)}) - y^{(i)} \right] x_j^{(i)}
 \end{aligned}$$

Multiclass Classification: One-vs-all

Eventhough Logistic Regression was created to solve binary classification problems it can also be used for more than two classes.

In this case, the problem is divided into $n+1$ subproblems ($n+1$ =number of highest class + 1).

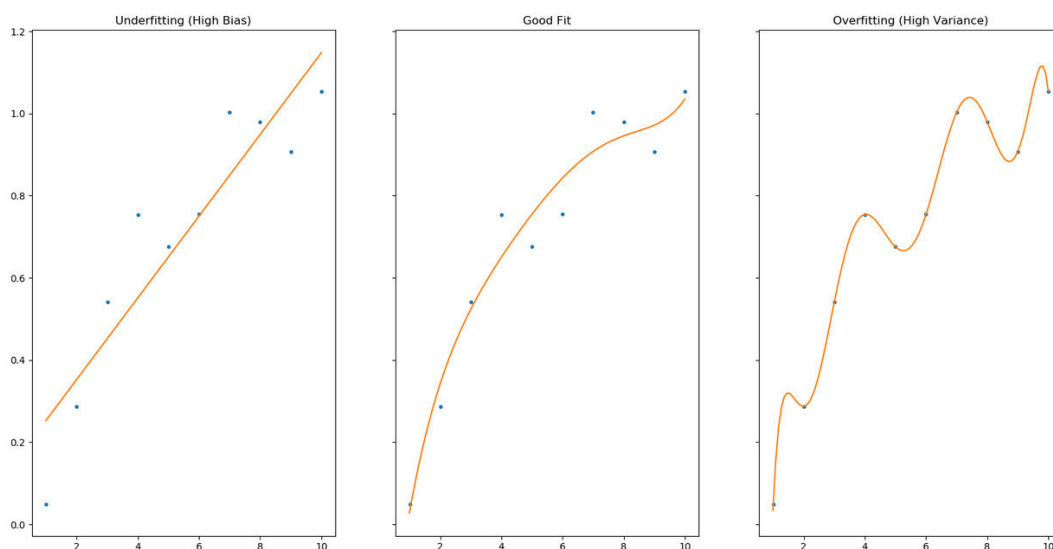


In each subproblem we predict the probability that y is a member of one of our classes.

We are basically choosing one class and putting all the others into a single class. We do this repeatedly until we went through each classes. Then we will find the result with the highest value and use this class as our prediction.

Regularized Logistic Regression

As you might know, regularization is a set of techniques designed to combat overfitting. Overfitting or also called high variance is the state when the model fits the available data but doesn't generalize well to unseen data. It is usually caused by an overcomplicated prediction function that creates lots of unnecessary curves and angles unrelated to the data.



There are two main options to address overfitting:

- Reducing the number of features
- Regularization

Manually reducing the number of features can be a tedious task because it often includes a lot of trial and error. Regularization, on the other hand, can happen automatically and has proven to be very reliable for lots of models over the years.

Logistic Regression can be regularized with the same techniques I explained when taking a look at [Linear Regression](#) – L1 and L2 Regularization.

Both techniques work by reducing the weights of the model by increasing their cost/loss. L1 Regularization takes the absolute values of all the weights and adds their sum to the loss. L2 Regularization sums up the squares instead of the absolute values.

For more information on the difference between L1 and L2 Regularization check out the following article:

- <http://www.chioka.in/differences-between-l1-and-l2-as-loss-function-and-regularization/>

L1 Regularization for Logistic Regression:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))] + \frac{\lambda}{2m} \sum_{j=1}^n |\theta_j|$$

L2 Regularization for Logistic Regression:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

As you can see above you not only can change from L1 to L2 Regularization but you can also increase or decrease the effect of regularization using λ .